

SECURITY AUDIT REPORT

for

OpenRouter

Prepared By: Xiaomi Huang

PeckShield

June 9, 2026

Document Properties

Client	OpenRouter
Title	Security Audit Report
Target	OpenRouter
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	June 9, 2026	Xuxian Jiang	Final Release
1.0-rc1	June 7, 2026	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction **4**
- 1.1 About OpenRouter . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 7

2 Findings **9**
- 2.1 Summary . . . 9
- 2.2 Key Findings . . . 10

3 Detailed Results **11**
- 3.1 Revisited Fund Rescue Logic in OpenRouter . . . 11
- 3.2 Strengthened User Input Validation . . . 12
- 3.3 Possibly Unlimited Return Data Copying . . . 13
- 3.4 Improved NatSpec Comments in Action Struct . . . 15

4 Conclusion **17**

References **18**

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the `OpenRouter` protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About OpenRouter

`OpenRouter` is a `Bungee` on-chain executor for swap, bridge, and swap-and-bridge routes. It combines structured entrypoints (`swap`, `swapAndBridge`, `bridge`) with a generic `performActions` loop for multi-step flows that need returndata splicing. The basic information of audited contracts is as follows:

Table 1.1: Basic Information of OpenRouter

Item	Description
Name	OpenRouter
Type	Solidity
Language	EVM
Audit Method	Whitebox
Latest Audit Report	June 9, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in this audit.

- https://github.com/SocketDotTech/openrouter.git (c84de29)

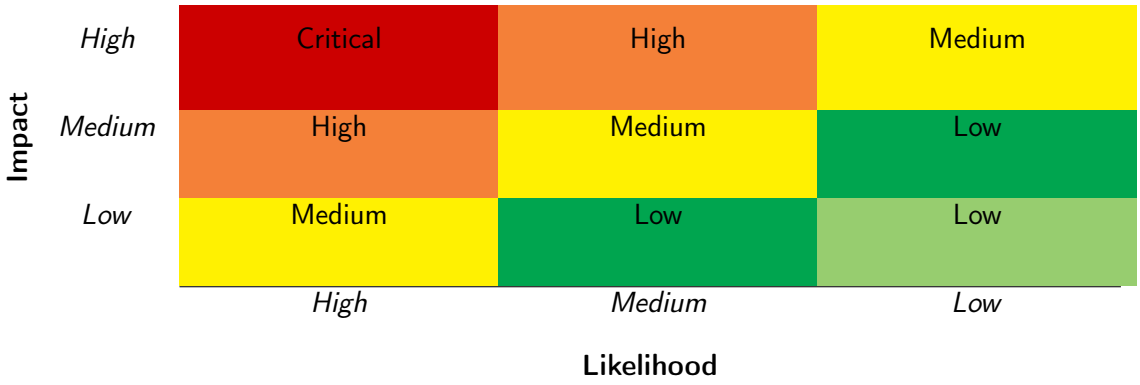
And here is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/SocketDotTech/openrouter.git (2ad98a4)

1.2 About PeckShield

PeckShield Inc. [7] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification



1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [6]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact, and can be accordingly classified into four categories, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [5], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings. Moreover, in case there is an issue that may affect an active protocol that has been deployed, the public version of this report may omit such issue, but will be amended with full details right after the affected protocol is upgraded with respective fixes.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 Findings

2.1 Summary

Here is a summary of our findings after analyzing the `OpenRouter` implementations. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	0	
Low	4	■■■■
Informational	0	
Total	4	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 4 low-severity vulnerabilities.

Table 2.1: Key Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Revisited Fund Rescue Logic in OpenRouter	Business Logic	Resolved
PVE-002	Low	Strengthened User Input Validation	Coding Practices	Resolved
PVE-003	Low	Possibly Unlimited Return Data Copying	Coding Practices	Resolved
PVE-004	Low	Improved NatSpec Comments in Action Struct	Coding Practices	Resolved

Beside the identified issues, we emphasize that for any user-facing applications and services, it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms should kick in at the very moment when the contracts are being deployed on mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 Revisited Fund Rescue Logic in OpenRouter

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `OpenRouter`
- Category: Business Logic [4]
- CWE subcategory: CWE-837 [2]

Description

The `OpenRouter` contract has a built-in mechanism to rescue funds from the contract if they are locked by mistake. This mechanism is mainly implemented in the `rescueFunds()` function. Our analysis shows that the presence of this function is largely unnecessary as it can be repurposed from other public functions e.g., `performActions()`.

In the following, we show the implementation of the related `rescueFunds()` routine. Its logic is rather straightforward in calling the library function to rescue the funds. Note that the `OpenRouter` contract is designed with the assumption without holding durable funds or taking approval from users. With that, the fund rescue mechanism is legitimate. However, our analysis shows that other functions can be repurposed for the same need. For example, the `performActions()` function lets any caller make the router call arbitrary contracts. If the router holds `ERC20s`, native `ETH`, bridged refunds, swap dust, rebates, or protocol refunds, a public caller can move or approve those assets through modular actions before owner rescue. Also, both `swap()` and `bridge()` functions can be repurposed to approve the `OpenRouter` spending of any `ERC20` tokens to an arbitrary third-party account.

```
762     /**
763      * @notice Rescues funds from the contract if they are locked by mistake.
764      * @param token The address of the token contract.
765      * @param rescueTo The address where rescued tokens need to be sent.
766      * @param amount The amount of tokens to be rescued.
767      */
768     function rescueFunds(address token, address rescueTo, uint256 amount) external
            onlyRole(RESCUE_ROLE) {
```

```
769         RescueFundsLib.rescueFunds(token, rescueTo, amount);
770     }
```

Listing 3.1: `OpenRouter::rescueFunds()`

Recommendation Revisit the above design in removing the dedicated `rescueFunds()` function. Its removal also makes the parent contract of `AccessControl` and the `owner` account not needed.

Status The issue has been resolved by this commit: `e5ef183`.

3.2 Strengthened User Input Validation

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `OpenRouter`
- Category: Coding Practices [3]
- CWE subcategory: CWE-1099 [1]

Description

As mentioned earlier, the `OpenRouter` contract combines structured entrypoints (`swap()`, `swapAndBridge()`, `bridge()`) with a generic `performActions()` loop for multi-step flows that need returndata splicing. Our analysis shows these structured entrypoints may be strengthened to better validate user inputs that are inherently untrusted.

In the following, we use the `swap()` function as an example and show below its code snippet that validates the user input. While the input parameters are validated to ensure the user input is sound, the validation can be improved to ensure the input token amount is also non-zero. Apparently, the swap of zero input token amount is not meaningful. Note the same issue also affects other routines, including `swapAndBridge()` and `bridge()`.

In addition, the `OpenRouter` contract may support the collection of swap/bridge fee via the above structured entrypoints. And the fee collection is deducted either from the input token or output token. Our analysis shows the use of `unchecked` primitive to depreciate the built-in arithmetic underflow detection. Such use is considered unsafe and needs to be revisited.

```
391     function swap(
392         bytes32 quoteId,
393         uint256 flags,
394         InputData calldata input,
395         FeeData calldata fee,
396         SwapData calldata swapData,
397         bytes calldata swapCallData,
398         address receiver
399     ) external payable returns (uint256 finalAmount) {
```

```
400         if (
401             input.user == address(0)  input.inputToken == address(0)  swapData.target ==
                    address(0)
402                 receiver == address(0)
403         ) {
404             revert InvalidExecution();
405         }
406         ...
407     }
```

Listing 3.2: `OpenRouter::swap()`

Recommendation Revisit the above-mentioned routines to strengthen the user input validation.

Status The issue has been resolved by this commit: `c836e47`.

3.3 Possibly Unlimited Return Data Copying

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `OpenRouter`
- Category: Coding Practices [3]
- CWE subcategory: CWE-1099 [1]

Description

To optimize the gas cost, `OpenRouter` makes extensive use of low-level calls when interacting with external contracts. With that, it inevitably needs to decode and return associated error messages from these low-level calls. One common caveat when handling the low-level error messages is the so-called gas exhaustion issue. Specifically, if there is a need to copy an untrusted contract's return data into memory without a size limit, a malicious contract can return a massive byte array. This forces to spend all its execution gas on memory expansion costs, causing the transaction to revert.

With that, we suggest to limit the error message size that may be returned from the low-level call. (A reasonable error message size is 128.) By doing so, we can still provide enough context for debugging while capping memory costs.

In the following, we show the implementation of an example routine, i.e., `_execCallCalldata()`. As the name indicates, it is used to execute a low-level call from the provided `calldata`. The blind copy of all return value (line 735) for a failed low-level call may be abused to expand memory costs. Note this issue also affects other routines, including `_performActions()` and `_execCall()`.

```
717     function _execCallCalldata(address target, uint256 value, bytes calldata data, bool
            storeResult)
718         internal
```

```
719         returns (bytes memory ret)
720     {
721         bool success;
722         assembly ("memory-safe") {
723             let ptr := mload(0x40)
724             calldatacopy(ptr, data.offset, data.length) // copy calldata slice to fresh
                    memory (avoids redundant memory alloc)
725             mstore(0x40, and(add(add(ptr, data.length), 0x1f), not(0x1f))) // advance
                    free pointer to next 32-byte boundary
726             success := call(gas(), target, value, ptr, data.length, 0, 0)
727         }
728
729         if (!success || storeResult) {
730             assembly ("memory-safe") {
731                 // prep and return revert data
732                 let returnDataSize := returndatasize()
733                 ret := mload(0x40)
734                 mstore(ret, returnDataSize) // write length prefix on free-mem pointer
735                 returndatacopy(add(ret, 0x20), 0, returnDataSize) // copy returndata
                        after length
736                 mstore(0x40, and(add(add(add(ret, 0x20), returnDataSize), 0x1f), not(0
                        x1f))) // bump free pointer
737             }
738             if (!success) {
739                 assembly ("memory-safe") {
740                     revert(add(ret, 0x20), mload(ret)) // bubble up the raw revert
                            payload
741                 }
742             }
743         }
744     }
```

Listing 3.3: `OpenRouter::_execCallCalldata()`

Recommendation Revise the above-mentioned routines to ensure the returned error messages are capped.

Status The issue has been resolved. Specifically, the call targets are backend-curated DEX/bridge contracts, not arbitrary user input, so returndata bombing is not considered a practical risk. Capping revert data would add gas and complexity for a less likely case the team decides not to handle on-chain.

3.4 Improved NatSpec Comments in Action Struct

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `OpenRouter`
- Category: Coding Practices [3]
- CWE subcategory: CWE-1099 [1]

Description

The `OpenRouter` protocol is well-documented with the extensive use of `NatSpec` comments to provide rich documentation for functions, return variables and others. In the process of analyzing current `NatSpec` comments, we notice the presence of an inconsistency with the code implementation in the defined `Action` struct used extensively in the generic `performActions()` routine.

```
59    struct Action {
60        /// @dev Packed call metadata. Decode with masks/shifts below; encode with
61        ///      `callType | (storeResult ? 1 << 8 : 0) | (uint160(target) << 16)`.
62        ///
63        /// Bit layout (least significant bits first):
64        ///   bits 255..160 : reserved (0)
65        ///   bits 159..16  : target address (uint160, left-aligned in this field)
66        ///   bit 8         : storeResult - when set, returndata is saved to `results[i
              ]`
67        ///                   even on success so later actions can splice from it
68        ///   bits 7..3     : reserved (0)
69        ///   bits 2..0     : CallType - CALL (0), STATICCALL (1), CALL_WITH_NATIVE (2)
70        ///
71        /// CALL_WITH_NATIVE: first 32 bytes of `data` are forwarded as `msg.value`;
72        /// the remaining bytes are the call payload.
73        uint256 actionInfo;
74        /// @dev Calldata passed to the target. Splices from `splices[]` overwrite byte
75        /// ranges in a mutable memory copy before the external call runs.
76        bytes data;
77        /// @dev Packed splice descriptors applied to `data` before the call.
78        /// Each entry is one `uint256` with four uint64 fields (see layout below).
79        /// Encode with `packSpliceInfo` in `scripts/e2e/utils/modularActionsBuilder/
              index.js`.
80        ///
81        /// Per-entry bit layout (least significant bits first):
82        ///   bits 255..192 : length - number of bytes to copy (must be > 0)
83        ///   bits 191..128 : dstOffset - byte offset into this action's `data` payload
84        ///                   (skips the bytes-array length word; for CALL_WITH_NATIVE,
85        ///                   offset 0 is the value word, offset 32 is payload start)
86        ///   bits 127..64  : srcOffset - byte offset into `results[sourceActionIndex]`
87        ///                   payload (same length-prefix convention)
88        ///   bits 63..0    : sourceActionIndex - index of a prior action (< current
              index)
89        ///
```

```
90          /// Packing formula:
91          ///   sourceActionIndex | (srcOffset << 64) | (dstOffset << 128) | (length <<
                192)
92          ///
93          /// The source action must have bit 8 set in `actionInfo` (storeResult); the JS
94          /// builder sets this automatically when a splice references that action.
95          uint256[] splices;
96      }
```

Listing 3.4: The `Action` Struct

To elaborate, we show above the full definition of the `Action` struct with three field members, i.e., `actionInfo`, `data`, and `splices`. An inconsistency comes from the first member field `actionInfo`. In particular, it has an embedded `target address` field that occupies the bits of `159..16` from the accompanying `NatSpec` comment. However, the actual bits for `target address` should be `175..16` since an address has `160` bits in total.

Recommendation Remove the inconsistency among the identified misleading `NatSpec` comment.

Status The issue has been resolved by this commit: `968594f`.

PeckShield

4 Conclusion

In this audit, we have analyzed the design and implementation of the `OpenRouter` protocol, which is a `Bungee` on-chain executor for swap, bridge, and swap-and-bridge routes. It combines structured entrypoints (`swap`, `swapAndBridge`, `bridge`) with a generic `performActions` loop for multi-step flows that need returndata splicing. The current code base is well structured and neatly organized. Those identified issues are promptly confirmed and addressed.

Meanwhile, we need to emphasize that smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1099: Inconsistent Naming Conventions for Identifiers. https://cwe.mitre.org/data/definitions/1099.html.

[2] MITRE. CWE-837: Improper Enforcement of a Single, Unique Action. https://cwe.mitre.org/data/definitions/837.html.

[3] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[4] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[5] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[6] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[7] PeckShield. PeckShield Inc. https://www.peckshield.com.